Refactoring Test Smells: A Perspective from Open-Source Developers

Elvys Soares*
Federal University of Pernambuco
Recife, Brazil
eas5@cin.ufpe.br

Márcio Ribeiro
Federal University of Alagoas
Maceió, Brazil
marcio@ic.ufal.br

Guilherme Amaral
Federal University of Alagoas
Maceió, Brazil
gvma@ic.ufal.br

Rohit Gheyi
Federal University of Campina Grande
Campina Grande, Brazil
rohit@dsc.ufcg.edu.br

Leo Fernandes
Federal Institute of Alagoas
Rio Largo, Brazil
leonardo.fernandes@ifal.edu.br

Alessandro Garcia
Pontifical Catholic University of Rio de Janeiro
Rio de Janeiro, Brazil
afgarcia@inf.puc-rio.br

Baldoino Fonseca
Federal University of Alagoas
Maceió, Brazil
baldoino@ic.ufal.br

André Santos
Federal University of Pernambuco
Recife, Brazil
alms@cin.ufpe.br

ABSTRACT

Test smells are symptoms in the test code that indicate possible design or implementation problems. Their presence, along with their harmfulness, has already been demonstrated by previous researches. However, we do not know to what extent developers acknowledge the presence of test smells and how to refactor existing code to eliminate them in practice. This study aims to assess open-source developers' awareness about the existence of test smells and their refactoring strategies. We conducted a mixed-method study with two parts: (i) a survey with 73 experienced open-source developers to assess their preference and motivation to choose between 10 different smelly test code samples, found in 272 open-source projects, and their refactored versions; and (ii) the submission of 50 pull requests to assess developers' acceptance of the proposed refactorings. As a result, most surveyed developers preferred the refactored proposal for 78% of the investigated test smells, and the pull requests had an average acceptance of 75% among respondents. Additionally, we were able to provide empiric validation for literature-proposed refactoring strategies. This study demonstrates that although not always using the academic terminology, developers acknowledge both the negative impact of test smells presence and most of the literature's proposals for their removal.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**; **Empirical software validation**.

*The author is also affiliated to the Federal Institute of Alagoas, Maceió, Brazil, available at elvys.soares@ifal.edu.br

SAST '20, October 20–21, 2020, Natal, Brazil

ACM ISBN 978-1-4503-8755-2/20/09…$15.00
https://doi.org/10.1145/3425174.3425212

KEYWORDS

Test Smells, Refactoring, Open Source, Validation

ACM Reference Format:
Elvys Soares, Márcio Ribeiro, Guilherme Amaral, Rohit Gheyi, Leo Fernandes, Alessandro Garcia, Baldoino Fonseca, and André Santos. 2020. Refactoring Test Smells: A Perspective from Open-Source Developers. In *5th Brazilian Symposium on Systematic and Automated Software Testing (SAST '20), October 20–21, 2020, Natal, Brazil.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3425174.3425212

1 INTRODUCTION

Automated software testing and the development of test scripts are frequent in the software industry [6]. A wide range of tools and frameworks for these activities, whether in support of creating automatic tests or the complete automatic generation of these, is already available [3, 9]. Automated test suites with high internal quality facilitate maintenance activities, such as code understanding and regression testing. However, the test code's development is tedious, error-prone, and requires a significant initial investment [1].

Poorly designed test code is frequent in practice [1], and the symptoms that indicate possible design problems in the test code are called bad test smells, or just test smells [6, 18]. These smells are not the same as those defined by Fowler [2] for the production code. As they are based on test cases organization, implementation, and interaction with one another, their corresponding refactoring operations generally differ from production code.

As a motivating example, Figure 1 presents a code snippet[1] extracted from the core module of the AssertJ project, self-described as *"a set of strongly-typed assertions to use for unit testing with any test automation framework that is very close to plain English"*. This test intends to verify a specific error and its specific properties. One can note the presence of a `try/catch` structure in the test and commands on lines 9 and 11 to handle the test outcome manually. According to Peruma et al. [11], manually passing or failing a test

[1]Original source code available at https://git.io/JJ1gA

depending on whether the production code throws an exception to a custom exception handling code in a test is expendable once the test framework already provides exception handling features to that end.

```
@Test
public void should_fail_if_bytes_are_equal() {
  AssertionInfo info = someInfo();
  try {
    booleans.assertNotEqual(info, TRUE, true);
  } catch(AssertionError e) {
    verify(failures).failure(info,
        shouldNotBeEqual(TRUE, true));
    return;
  }
  failBecauseExpectedAssertionErrorWasNotThrown();
}
```

Figure 1: Example of the Exception Handling test smell

Later, we observed a test code refactoring[2] using both AssertJ new features and Java 8 lambda expressions that made the exception test more transparent and eliminated the previous need for manually dealing with errors, and manually passing or failing the test. Figure 2 presents the refactoring performed by the AssertJ team, eliminating the fragility of the original design.

```
@Test
void should_fail_if_bytes_are_equal() {
  AssertionInfo info = someInfo();
  Throwable error = catchThrowable(() ->
      booleans.assertNotEqual(info, TRUE, true));
  assertThat(error).isInstanceOf(AssertionError.class);
  verify(failures).failure(info,
      shouldNotBeEqual(TRUE, true));
}
```

Figure 2: Refactoring the Exception Handling test smell

The provided example reflects what a test smell means: a symptom in the test code that may (or may not) indicate a design problem. The subsequent refactoring action demonstrated awareness of the design problem and implemented a coding strategy to eliminate it.

Although numerous previous studies have already demonstrated the presence of test smells in test suites of open-source projects, as well as their negative impact on code maintenance and understanding activities [1, 9–11, 19], there is a lack of studies to assess the awareness of open-source developers about the existence of tests smells and their possible solutions.

Thus, this study presents two empirical investigations that aim to answer the following research questions:

[2]Refactored source code available at https://git.io/JJ1Vn

- **RQ_1: To what extent test smells are relevant to open-source developers?**
- **RQ_2: To what extent open-source developers accept refactorings that remove test smells?**

The first empirical study is a survey with open-source developers to investigate **RQ_1** by considering questions such as *"Are open-source developers aware of test smells and recognize smelly test code?"* and *"Do they know how to prevent or refactor a test smell?"*. This study consisted of showing smelly test code snippets that we gathered from open-source projects, along with an alternate refactored version of the same test code. Then we asked developers to choose which version they preferred and justify their answers.

The second empirical study aimed at answering **RQ_2** and questions such as *"Do they acknowledge the existence of test smells in their projects?"* consists of submitting contributions (Pull Requests) with test code refactorings to active open-source projects and collect whether the developers of such projects will accept the contributions, along with considerations to any possible discussion topics that may emerge from the contributions. We have sent Pull Requests to 50 different open-source projects and, among respondents, we achieved a 75% acceptance rate, having most contributions being accepted with very little further discussions.

In summary, this paper provides the following contributions:

- A survey with experienced open-source developers that assessed their impressions about test smells and their refactoring strategies in Section 3;
- An empirical study based on pull requests to better understand the acceptance level of open-source developers to refactorings of test smells in their projects, along with an empirical validation of 10 literature-proposed refactoring strategies in Section 4;

2 REFACTORING TEST SMELLS

We now present a brief explanation of the test smells we used in this study, the strategies to eliminate them and the related code snippets (simplified as toy examples). According to the test smell distribution presented by Peruma et al. [11], these test smells figure between the highest available ones on open-source projects:

Original

```
boolean b = MyComponent.isB();
@Test
public void test() {
    if (b) {
        MyClass c = new MyClass();
        assertTrue(c.isA());
    }
}
```

Refactored

```
boolean b = MyComponent.isB();
@Test
public void test() {
    assertTrue(b);
    MyClass c = new MyClass();
    assertTrue(c.isA());
}
```

Figure 3: Conditional Test Logic

Conditional Test Logic: Defined by Meszaros [6], this test smell occurs when the test depends on a condition. It is a dangerous design since a test method may result in a passed status without

ever having asserted a unit result. Palomba et al. [10] propose refactoring this test smell with an *"Extract Method"* [2] operation, which we believe to be more suitable in cases of nested loops and decisions. As our example uses a decision structure to verify a precondition, we added an assumption to the precondition, which will make the test to be ignored if not met. Figure 3 presents both the original and the refactored versions.

Assertion Roulette: It is defined by van Deursen et al. [18] as a collection of unexplained assertions in a single test method that makes it difficult to trace which exact assertion had a problem in the event of test failure. This test smell has the original refactoring suggestion of *"Add Assertion Explanation"*, and an alternative suggestion to break up the test into a suite of *"Single-Condition Tests"* proposed by Meszaros [6]. As a single-condition test is actually a good programming practice that increases code coverage, we opted to use this refactoring strategy. Figure 4 presents the original program with the Assertion Roulette and its refactored version.

Original

```
1.  private MyClass c;
2.  @Before
3.  public void setup() {
4.      c = new MyClass();
5.  }
6.  @Test
7.  public void test() {
8.      assertTrue(c.isA());
9.      assertTrue(c.isB());
10.     assertTrue(c.isC());
11. }
```

Refactored

```
1.  private MyClass c;
2.  @Before
3.  public void setup() {
4.      c = new MyClass();
5.  }
6.  @Test
7.  public void testA() {
8.      assertTrue(c.isA());
9.  }
10. @Test
11. public void testB() {
12.     assertTrue(c.isB());
13. }
14. @Test
15. public void testC() {
16.     assertTrue(c.isC());
17. }
```

Figure 4: Assertion Roulette

Exception Handling: According to Peruma et al. [11], this smell occurs when the test manually handles both exceptions and test outcome and its frequency rivals with Assertion Roulette on Java projects. Figure 5 presents the first case of this test smell, along with its refactoring, when properties after an exception must be verified. Figure 6 presents the case 2 of the Exception Handling test smell, that occurs when an exception message needs to be verified along with an alternate version that uses JUnit framework features. Figure 7 presents case 3, along with an alternate refactored option, which happens when some final steps must be executed independently of the test result or the existence of any exceptions. Figure 8 presents case 4, which tests for a specific exception being raised. The JUnit framework recommends the use of `@Test(expected=MyException.class)` annotation in this case.

Original

```
1.  @Test
2.  public void test() {
3.      MyClass c = new MyClass();
4.      try {
5.          c.throwSomeException();
6.          Assert.fail("Exception not thrown");
7.      } catch (SomeException e) {
8.          assertTrue(c.isA());
9.          assertTrue(c.isB());
10.         assertTrue(c.isC());
11.     }
12. }
```

Refactored

```
1.  @Test
2.  public void test() {
3.      MyClass c = new MyClass();
4.      Throwable t = catchThrowable(() -> c.throwSomeException());
5.      assertThat(t).isInstanceOf(SomeException.class);
6.      assertTrue(c.isA());
7.      assertTrue(c.isB());
8.      assertTrue(c.isC());
9.  }
```

Figure 5: Case 1 of Exception Handling

Original

```
1.  @Test
2.  public void test() throws Exception {
3.      try {
4.          MyClass c = new MyClass();
5.          c.throwSomeException();
6.          failBecauseExceptionWasNotThrown(SomeException.class);
7.      } catch (SomeException e) {
8.          assertThat(e.getMessage()).isEqualTo("Error message");
9.      }
10. }
```

Refactored

```
1.  @Rule
2.  SomeException e = SomeException.none();
3.  @Test
4.  public void test() {
5.      e.expect(SomeException.class);
6.      e.expectMessage("Error message");
7.      MyClass c = new MyClass();
8.      c.throwSomeException();
9.  }
```

Figure 6: Case 2 of Exception Handling

Original

```
@Test
public void test() throws Exception {
    MyClass c = new MyClass();
    c.initialize();
    try {
        assertTrue(c.isA());
    } finally {
        c.finalize();
    }
}
```

Refactored

```
private MyClass c;
@Before
public void setup() throws Exception {
    c = new MyClass();
    c.initialize();
}
@Test
public void test() throws Exception {
    assertTrue(c.isA());
}
@After
public void tearDown() throws Exception {
    c.finalize();
}
```

Figure 7: Case 3 of Exception Handling

Original

```
@Test
public void test(){
  MyClass c = new MyClass();
  try{
    c.throwMyException();
    fail();
  } catch(MyException e){
    // expected
  }
}
```

Refactored

```
@Test(expected=MyException.class)
public void test(){
  MyClass c = new MyClass();
  c.throwMyException();
}
```

Figure 8: Case 4 of Exception Handling

Sensitive Equality: van Deursen et al. [18] state the agility and ease to write equality checks using the `toString()` method, being a frequent alternative to calculate a result value and map it to a sequence to compare to a literal string representing the expected value. However, these tests can depend on many irrelevant details, such as commas, quotes, and spaces. And whenever the `toString()` method for an object is changed, all related tests will start to fail. The proposed solution to this test smell is to replace the equality checks that use the `toString()` method with real equality checks using the *"Introduce Equality Method"* refactoring strategy [18]. Figure 9 presents an example of Sensitive Equality, along with a refactored alternative.

Original

```
@Test
public void test() {
    MyClass c = new MyClass();
    c.performAction();
    // Compare result using toString() method
    assertEquals(c.toString(), "Expected");
}
```

Refactored

```
@Test
public void test() {
    MyClass c = new MyClass();
    c.performAction();
    // Compare result using an equality method
    assertEquals(c.equalityCheck(), "Expected");
}
```

Figure 9: Sensitive Equality

Original

```
@Test
public void test() {
    Map<String,Integer> myMap = new MyMap<String,Integer>();
    String key = "";
    myMap.put(key, 3653);
    assertThat(myMap.get(key)).isEqualTo(3653);
    myMap.replace(key, 1);
    assertThat(myMap.get(key)).isEqualTo(1);
}
```

Refactored

```
final Integer FIRST_VALUE = 1;
final Integer LAST_VALUE = 3653;
@Test
public void test() {
    Map<String,Integer> myMap = new MyMap<String,Integer>();
    String key = "";
    myMap.put(key, LAST_VALUE);
    assertThat(myMap.get(key)).isEqualTo(LAST_VALUE);
    myMap.replace(key, FIRST_VALUE);
    assertThat(myMap.get(key)).isEqualTo(FIRST_VALUE);
}
```

Figure 10: Magic Number Test

Magic Number Test: Proposed by Peruma et al. [11], this smell occurs when the test method contains undocumented numeric

literals with no clear meaning (magic values). To refactor this test smell, the authors state that these numeric values can be replaced with constants or variables, thus providing a descriptive name for the value. This smell is presented in Figure 10.

Resource Optimism: According to van Deursen et al. [18], this smell happens when test methods make optimistic assumptions about the existence or the state of external resources like files and databases. Such assumptions may cause non deterministic behavior on the outcome and can be refactored through the *"Setup External Resource"*, also defined before [18]. Figure 11 presents our example of Resource Optimism and the corresponding refactoring operation.

Original

```
@Test
public void test() throws Exception {
    ExternalResource r = new ExternalResource();
    MyClass c = new MyClass();
    c.setA(r.performAction());
    assertTrue(c.isA());
}
```

Refactored

```
private ExternalResource r;
private MyClass c;
@Before
public void setup() throws Exception {
    r = new ExternalResource();
    c = new MyClass();
}
@Test
public void test() throws Exception {
    c.setA(r.performAction());
    assertTrue(c.isA());
}
```

Figure 11: Resource Optimism

Test Code Duplication: Also proposed van Deursen et al. [18], this test smell normally identifies classes that contain test methods with repeated test code steps. Typically, parts that set up tests fixtures are normally related to this test smell. The authors state that, for cases where the duplicated code is in the same class, this test smell can be eliminated using *"Extract Method"*, defined by Fowler [2], as it is for normal code duplication. Figure 12 presents our example, as well as its refactored version.

The next sections present the empirical investigations we performed in this study. The replication package containing all the raw data of both investigations can be found at the companion website[3].

3 SURVEY WITH DEVELOPERS

In this section, we present our first empirical study performed with open-source developers.

[3]https://bit.ly/2EeL7Io

Original

```
MyClass c = new MyClass();
@Test
public void test1() {
    assertTrue(c.isA());
    assertTrue(c.isB());
}
@Test
public void test2() {
    assertTrue(c.isA());
    assertTrue(c.isB());
    assertTrue(c.isC());
}
```

Refactored

```
MyClass c = new MyClass();
public void commonAsserts() {
    assertTrue(c.isA());
    assertTrue(c.isB());
}
@Test
public void test1() {
    commonAsserts();
}
@Test
public void test2() {
    commonAsserts();
    assertTrue(c.isC());
}
```

Figure 12: Test Code Duplication

3.1 Planning

This survey aimed to analyze implementation preferences concerning the presence of test smells and their refactoring identification, from the viewpoint of open-source developers, as stated by **RQ_1: To what extent test smells are relevant to open-source developers?**

The general idea was to ask developers for their preference between two implementations of equivalent unit tests. One of these implementations was always an original (smelly) code snippet, and the other one was a refactored alternative, following the refactoring principles of Murphy-Hill, Parnin, and Black [7].

3.2 Settings

To assemble the survey, we needed a list of test smells from open-source projects. As there is no public list of such smells, we had to search and use a tool to analyze projects from public repositories.

Although the most comprehensive survey to the date [3] lists 12 tools that handle test smells, the most recent and complete one is the TSDetect [11] tool proposed by Peruma et al. [11]. It detects 19 different test smells and analyzes JUnit test cases, creating detailed execution logs. Using the GitHub search, we retrieved the projects list by classifying Java projects in descending order of stars and executed the TSDetect [11] in the listed results. According to the tool execution logs, we achieved a test smell distribution result consistent with that reported by Peruma et al. [11] with the first 272 results.

3.3 Design

Intending to have as many respondents as possible, we defined the survey to take an average of 10 minutes of the developers' time. This way, we searched for suitable test methods that (i) had few lines of code and could be displayed along with the refactored version without the need for scrolling the screen, and (ii) had an identifiable test smell, albeit we did not stress if the code had any problem, nor highlighted it in any way.

The pilot execution of the survey revealed a significant concern about the displayed code snippets: developers kept trying to understand project specificities, taking much longer than anticipated, and not noticing the problems we wanted them to analyze. To mitigate this issue, we reduced the code snippets to toy examples, as presented in the code examples from Section 2.

Through scripts, we retrieved the contributors of open-source Java projects who had public contact info available and contributed to test activities within their projects. Our scripts analyzed about 600 projects, and a total of 2,011 invitation emails were sent.

Along with questions concerning demographics and the developers' expertise in software testing, we presented 9 of the test smells examples presented in Section 2, only excluding the Exception Handling 4, which is a simple framework usage, as we intended the survey to last the least possible time. The test smells, and their refactoring options were displayed as A/B questions, which were presented in random order to every developer, who could choose from the following list of options: *(i) I strongly prefer A, (ii) I prefer A, (iii) Indifferent, (iv) I strongly prefer B, (v) I prefer B, and (vi) I do not know.* All questions were mandatory and provided an optional field for the developers to justify their answers.

3.4 Results and Discussion

We performed the survey in June 2020, achieving a response rate of 3.63% with the total number of 73 responses. Concerning the demographics, area of work, and expertise questions, we had participants working in 26 countries, 89% of them defined their primary area of work as the industry (over academia), 68.5% declared to have 10+ years of experience with software testing. This number rises to about 87.5% if we consider 5+ years of experience. The average of their daily time spent on testing activities is about 41%. Concerning the A/B questions, Table 1 presents the obtained results, in percentage.

Test Smell	Original	Refactored	Other
Assertion Roulette	12.3%	65.8%	21.9%
Conditional Test Logic	12.3%	64.4%	23.3%
Exception Handling 1	24.7%	68.5%	6.8%
Exception Handling 2	38.4%	37.0%	24.7%
Exception Handling 3	26.0%	68.5%	5.5%
Magic Number Test	31.5%	54.8%	13.7%
Resource Optimism	27.4%	49.3%	23.3%
Sensitive Equality	37.0%	32.9%	30.1%
Test Code Duplication	31.5%	49.3%	19.2%

Table 1: Survey results

In order to better interpret the obtained results, both numeric and comment analyses are fundamental. While the numeric results indicate whether developers accept a proposed refactoring method, the comment analysis tells us if they are doing so because they correctly recognize the possible flaw in the test code.

The Assertion Roulette result showed that although the provided refactoring was not the originally proposed one, where van Deursen et al. [18] suggests commented assertions, most developers prefer to separate assertions into single test cases. The provided comments reinforce their choices: *"It tests different methods on a class so if one of them fails it is easy to see which one by looking at test results"* and *"(refactored) is more code, but it will give more fine-grained reporting when there is a failure."*.

The Conditional Test Logic had results consistent with the Assertion Roulette. Although most developers claimed both tests not to be equivalent because the original test would pass without testing all scenarios, they seemed to understand the intent of the condition: *"(Original) suggests that b is somehow an assumption that must be satisfied for executing the rest of the test (I would do that differently, though)"*.

The first and third cases of Exception Handling had their refactored versions preferred by most developers. The most common argument was using the test framework resources and the improvements in code readability and maintainability. Regarding the second case of the Exception Handling test smell, where an exception message needed to be tested, although comments about the existence of try/catch blocks in test code followed the other Exception Handling cases, the provided refactoring implementation using the Rule annotation divided opinions. A frequently provided reason is given in the comment *"I am not familiar with this '@Rule'. The code looks prettier (which in general means more maintainable), but the ordering is a bit counter-intuitive"* reinforce our statement.

Developers well recognized the Magic Number Test Smell. Some of them argued that the original test code would be acceptable if the test method is unique. Comments like *"Using well-named constants enhances readability when comparing with magic numbers usage"* demonstrate that developers value code semantics.

Although the refactored version of the Resource Optimism test smell was the choice of almost half of the developers, most comments concerned *"Using a @Before method allows reuse by multiple tests."*, which indirectly solves the test smell. Fewer comments stressed the need to have external resources instantiated separately from the test steps, as in *"Allows me to distinguish between failures in setting up tests vs. the actual test I want to perform"*.

Concerning the Sensitive Equality test smell, as the distribution of choices was slightly balanced between the options, it is possible to find (i) critics to the use of the `toString()` method: *"The 'toString' method is not always a reliable indicator of object state."*, (ii) critics to the equality check method: *"The equality check method seems to be created for the test: the test should test the real behavior of the object"* and (iii) critics to the example design: *"Semantics of equalityCheck are not at all obvious"*. The comments analysis demonstrates that most developers recognize the used of `toString()` as problematic, but disagree whether an equality check method is the best refactoring strategy. We also believe reducing a toy example could not evidence the test smell to all developers, causing them difficulties understanding the root problem. Maybe a more suitable example would make the 30% of developers who opted for "Indifferent" and " I do not know" to choose more accurate opinions.

Almost half of the developers selected the refactored Test Code Duplication. They highlight the need to avoid duplicate test code and good practices as the use of DRY[4] principles. The other groups

[4]The Don't Repeat Yourself principles were defined by Thomas and Hunt [16]

kept elaborating whether the use of an *"Extract Method"* [2] in such a small example would be over-engineering.

> **Summary:** Even when developers did not choose the refactored version, they correctly indicated the code fragility that motivates the test smell definition and sometimes the refactoring fragility. Therefore we conclude that open-source developers acknowledge test smells refactoring strategies, positively answering $\mathbf{RQ}_1$.

3.5 Threats to Validity

The primary construct validity threat is related to creating toy examples from both original and refactored code snippets, which may have resulted in confusing or over-engineered toy examples that could lead developers to misjudgment. Despite having the survey presentation page with highlights on the simplified, but real open-source examples, and the process of simplification we submitted the real code snippets to, we could find comments to answers considering the toy examples as literal ones. As internal threats, the code examples we collected from open-source projects in the early elaboration stage of the survey may not be the best representatives of the intended test smells and their refactoring strategies, or cover all possible test smells scenarios. The Exception Handling 4, presented in Section 2, which was not part of the survey, although being a relatively simple case, is an excellent example of a lack of complete coverage. As external threats, this study evaluated 7 test smells and their refactoring strategies. While broad surveys like Garousi and Küçük [3] and Peruma [13] have, combined, cataloged, and proposed more than 130 different test smells, generalizing our results to a larger group may not be a valid assumption.

4 SUBMISSION OF PULL REQUESTS

This section describes the second empirical investigation performed in this study, where contributions to open-source projects were made to assess developers' opinions about test smell refactorings.

4.1 Planning

Our second empirical study aims to analyze contributions to open source projects regarding the acceptance to test smell refactorings from the viewpoint of the project maintainers, as stated by $\mathbf{RQ}_2$: To what extent open-source developers accept refactorings that remove test smells?

We assume that the acceptance rate of the pull requests might corroborate that test smell refactoring is relevant. In this study, we also focused on the 7 test smells and 10 refactoring strategies presented in Section 2.

4.2 Settings

Starting from the TSDetect [11] execution logs acquired in the first study, we manually searched for test smells that fit the generic templates defined by the toy examples. Once a test smell was identified, the same refactoring strategy used in the developers' survey was applied.

As explained, for each system, TSDetect [11] reports test smells through its execution logs (Step 1). Because we execute TSDetect [11] considering the entire project test code, we could find several test smells (even the ones we are not considering in this study). This way, for each project, we randomly selected an identified test smell. The intention was to assess the developers' acceptance of that specific smell. Besides, to minimize bias on pull request acceptance from the same developer, we decided to submit only one contribution per project. Then, given the selected test smell, we applied a proper refactoring (see Section 2) and submitted a pull request (Step 2). Notice that we neither fix bugs nor submit new test cases. We focused on submitting that one specific test smell refactoring. In addition to sending the pull request, we also submitted a comment justifying the transformation that, unless stated differently by specific projects submission rules, followed the pattern of presenting the definition of the problem, the description of the refactoring strategy (proposed solution), and the before and after (code snippets) changes. When necessary, we discussed with the developers to defend our submission before deciding to accept or reject the pull request (Step 3).

4.3 Results and Discussion

Although we have submitted 50 pull requests, we were limited by the template models we defined in the first study. Also, many difficulties were encountered regarding dependencies on projects' environments, which forced us to abandon submitting some pull requests. As we explained, we submitted one pull request per project, which leads us to 50 open-source projects involved in this part of the study. Table 2 presents the pull requests submission distribution by test smell type.

Test Smell Type	Submitted	Accepted	Rejected
Assertion Roulette	8	50%	N/A
Resource Optimism	6	16%	33%
Conditional Test Logic	9	44%	N/A
Exception Handling (all)	12	50%	8%
Magic Number Test	8	12.5%	25%
Sensitive Equality	2	50%	50%
Test Code Duplication	5	20%	N/A
Total:	**50**		

Table 2: Pull Requests by Test Smell type

Until submitting this study, we received 18 acceptances, 6 rejections, and noticed 4 submission errors. The average time the respondent projects took to analyze a pull request was two business days. We had no projects that contested the submissions and required long, convincing dialogues. However, we had one occurrence (Conditional Test Logic) that investigated, on project details, the real necessity of the conditional test logic and ended up agreeing with the refactoring proposal.

Table 3 presents the pull requests we submitted in this study. It also provides additional project details and summarizes the developers' comments.

No	Project Name	SW Category	Test Smell Type	Status	Pull Request Comments
1	ffmpeg-cli-wrapper	App	Resource Optimism	Submission Error	"Wrong usage of project classes"
2	gumtree	Framework	Assertion Roulette	Open	
3	google-maps-services-java	Library	Exception Handling	Submission Error	"Wrong pull request format"
4	dropwizard	Framework	Conditional Test Logic	Accepted	"Thanks for your contribution"
5	docx4j	Library	Assertion Roulette	Open	
6	jvm-tools	App	Assertion Roulette	Accepted	No Comments
7	lavagna	App	Resource Optimism	Accepted	"Thanks for your contribution"
8	mango	Framework	Assertion Roulette	Open	
9	mybatis-3	Framework	Conditional Test Logic	Submission Error	"Wrong usage of project classes"
10	pi4j	Library	Exception Handling	Accepted	"Thanks for your contribution"
11	pippo	Framework	Conditional Test Logic	Accepted	"Thanks for your contribution"
12	Fenzo	Library	Exception Handling	Open	
13	fluent-validator	App	Assertion Roulette	Accepted	"Thanks for your contribution"
14	realm-android-adapters	Android App	Exception Handling	Open	
15	zxing-android-embedded	Library	Assertion Roulette	Open	
16	wiremock	App	Magic Number	Open	
17	gerrit-intellij-plugin	Plugin	Assertion Roulette	Accepted	"Thanks for your contribution"
18	JSCover	App	Exception Handling	Accepted	"Thanks for your contribution"
19	pac4j	App	Exception Handling	Accepted	"Thanks for your contribution"
20	disruptor	Library	Magic Number	Rejected	"Test is harder to understand"
21	hutool	Library	Assertion Roulette	Accepted	"Thanks for your contribution"
22	spring-boot-thin-launcher	Framework	Conditional Test Logic	Open	
23	assertj-core	Framework	Resource Optimism	Rejected	" Current test behavior preferred"
24	Apktool	App	Resource Optimism	Rejected	"Current test behavior preferred"
25	rest-client	App	Resource Optimism	Open	
26	bitcoinj	Library	Sensitive Equality	Rejected	"Wrong Java API usage"
27	RxCache	Library	Conditional Test Logic	Open	
28	blade	Framework	Magic Number	Open	
29	size-analyzer	Tool	Resource Optimism	Submission Error	"Wrong pull request format"
30	bt	Framework	Conditional Test Logic	Open	
31	RegexGenerator	Library	Test Code Duplication	Open	
32	bugsnag-android	Library	Conditional Test Logic	Accepted	"Thanks for your contribution"
33	bundletool	Tool	Conditional Test Logic	Accepted	"Thanks for your contribution"
34	cordova-android	Framework	Test Code Duplication	Open	
35	Conductor	Framework	Test Code Duplication	Open	
36	commonmark-java	Library	Test Code Duplication	Open	
37	tablesaw	Library	Sensitive Equality	Accepted	No Comments
38	webcam-capture	Library	Magic Number	Open	
39	unirest-java	Library	Magic Number	Accepted	"Thanks for your contribution"
40	testcontainers-java	Library	Magic Number	Rejected	"Test is harder to understand"
41	client_java	App	Exception Handling	Accepted	"Thanks for your contribution"
42	compile-testing	Tool	Exception Handling	Rejected	"Discouraged test framework feature"
43	scribejava	Library	Exception Handling	Accepted	"Unaware of test framework feature"
44	eclipse-collections	Framework	Conditional Test Logic	Open	"Rejected test framework feature"
45	spark	Library	Magic Number	Open	
46	sofa-bolt	Framework	Exception Handling	Open	
47	stetho	App	Magic Number	Open	
48	binding-collection-adapter	Library	Exception Handling	Accepted	No Comments
49	AndroidNetworkTools	Library	Test Code Duplication	Accepted	No Comments
50	java-dns-cache-manipulator	Library	Exception Handling	Open	

Table 3: Submitted pull requests

Regarding the pull requests' comments, it is vital to notice that most of the accepted pull requests were made with a simple *"Thanks for your contribution"* message or no message at all. From this fact, we understand that the developers accepted both the presence of the test smell and the refactoring strategy.

As to the rejected pull requests, four different test smells are on the list: Magic Number Test (2), Sensitive Equality (1), Resource Optimism (2) and Exception Handling (1).

In the rejected Magic Number Tests, the first developer claimed that *"Hard-coded numbers aren't automatically bad."* and *"This change makes the tests harder to understand."*, and closed the pull request after that. Many of the numeric literals inserted directly into the code are used, for example, to retrieve the first position of an array, or to assert the size of a known array. Such numbers are not difficult to read and maintain in the code, and this coding practice seems to be widely used in the analyzed open-source projects. Replacing them with constants or variables may add unnecessary text. The second developer argued that *"200 is a well-known status code"* and *" I don't think changing it would make any sense."*. Although we had accepted pull requests to this same case, well-known response codes may be an argument in favor of the magic numbers.

Other rejected pull request was related to the Sensitive Equality test smell. In this case, the developer arguments that *"The* `toString() / fromString()` *API is actually the recommended API to use if you deal with addresses."*. Indeed, if an API is widely used to retrieve a text value from an object using the `toString()` method, then defining a new method that always returns the same value as `toString()` may not be necessary.

Two rejected pull requests were related to the Resource Optimism test smell. Both developers did not want the failing assumption of the external resource, which was not the software unit under test, to ignore the test execution. They preferred the original code, which would fail if the resource is not available.

The last rejection is related to the Exception Throwing test smell. The response given by the developer was "*we actually discourage the use of* `@Test(expected = ...)` *in general.*" and "*a test that uses* `@Test(expected = ...)` *" will pass if any expression in the test throws an exception of the expected type, even if you only expect the exception from a particular call.* It is a good rationale but might apply mostly to integration tests, which are frequently populated with several steps and verifications.

The four submission errors were due to the lack of complete understanding of the project architecture and the software unit under test, or even the project rules for pull requests, which generated bad submissions.

Summary: Even when a pull request was refused, the discussion with developers showed that they correctly identify the general "issue" we were referring to and were able to discuss why the test smell did not apply to their projects. The 75% acceptance, mostly without discussion, leads us to conclude that open-source developers accept refactorings test smells in their projects, thus positively answering $\mathbf{RQ}_2$.

4.4 Threats to Validity

The construct validity relates to the refactoring templates presented in Section 2, which were not extended to all scenarios within a studied test smell type. A clear example of such an untreated template is the Conditional Test Logic cases that nest their assertions with repetition and decision structures and did not receive pull requests in this study. As internal validity, the submitted pull requests by test smell type may not be enough to generalize results per studied test smell. Concerning external validity, as we worked with almost the same test smells and refactorings we used in the survey with developers, this group of test smells may not be representative of generalizing results to the test smells we did not cover in this study.

5 RELATED WORK

Palomba et al. [8] performed a survey on the developer's perception of bad code smells. They showed production code snippets from three software systems to original and outside (students and industry) developers, asking them to indicate if they found any potential design problems and what nature and severity level they would classify the possible problems. As a result, they could divide the investigated code smells into categories related to code complexity and coding good-practices. Our study offers a similar perception by open-source developers, this time concerning test smells.

The study performed by Bavota et al. [1] demonstrated test smell distribution in software systems and whether their presence is harmful. As part of the investigation, they performed a controlled experiment involving 61 participants among students (freshers, bachelors, and masters) and industrial developers, which were asked to perform maintenance activities on smelly and refactored test code of two software systems. The study demonstrated the negative impact of test smells in program comprehension during maintenance activities. Our study uses both open-source developers and projects to survey on test smells perceptions.

Tufano et al. [17] investigation aimed at analyzing when test smells occur in source code, what their survivability is, and whether their presence is associated with the presence of design problems in production code (code smells). They collected the developers' perception of test smells in a study with 19 developers from Apache and Eclipse ecosystems. They demonstrated that (i) test smells have a long life cycle in software systems, and (ii) there are correlations between test and code smells. Our study extends the surveyed public in order to improve the accuracy of developers' perception.

Lambiase et al. [5] presented the DARTS (Detection And Refactoring of Test Smells) tool. It detects instances of three test smell types (General Fixture, Eager Test, and Lack of Cohesion of Test Methods) at the commit-level and enables their automated refactoring using the refactoring techniques defined by Meszaros [6]. The study proposes refactorings based on previously-proposed strategies. Our study validates the same strategies using developers' opinions.

A work in progress performed by Schvarcbacher et al. [14] aimed to assess the perception of developers on test smells in their codebase and which ones they would consider being more important. They present the developers' reactions to various instances of test smells pointed out by their detection tool, based on the TSDetect [11], in integration with the Better Code Hub (BCH).[5] Their

[5]https://bettercodehub.com/

results show that developers are only willing to remove a small portion of the found test smells, but did not present them with refactored alternatives to measure their acceptance.

The study performed by Spadini et al. [15] aimed at investigating the severity rating for four test smells and their perceived impact on test suite maintainability by the developers. They collected test smells from 1,500 open-source projects and used in-company developers' perceptions to calibrate the established severity thresholds for test smells. As results, they found that current detection rules for certain test smells are considered too strict by the developers and verified that their newly defined severity thresholds are in line with the participants' perception of how test smells impact the maintainability of a test suite.

Although researching refactoring operations, Peruma et al. [12] used a mining tool to detect refactoring operations from an existing dataset of unit test files and smells in 250 open-source Android Apps. Among the results, they could demonstrate that the types of refactorings applied to test and non-test files in Android apps are different and that there exist test smells and refactoring operations that co-occur frequently. More importantly, and following the results of our study, they could verify that developers apply refactorings for reasons other than the correction of smells, making the fixing of smells merely a byproduct.

Silva Junior et al. [4] carried out an expert survey to analyze the usage frequency of a set of test smells, aiming to understand whether test professionals non-intentionally insert test smells. Using an observation by case-control study design, they evaluated the answers of 60 industry developers on 14 test smells. As a result, they found that developers introduce test smells during their daily programming tasks, even when using their companies' standardized practices.

6 CONCLUSIONS

In this paper, we conducted two empirical studies regarding open-source developers' awareness about test smells and the refactoring strategies to mitigate them.

In the first study, we surveyed 73 developers, who analyzed 10 test smells from real open-source projects and their refactored versions. The answers and their justifications led us to conclude that even when developers disagree with the proposed refactorings, they are aware of the threats proposed by the smelly code. This study was also able to provide empirical validation to most of the literature-proposed refactoring strategies we used in the assessed test smells.

In the second study, we submitted 50 Pull Requests to open source projects whose tests had at least one of the 7 test smells we investigated in the previous study to verify if the developers accepted our contributions. The 75% acceptance rate among respondents indicates that developers are also prone to accept contributions that solve test smells in their test code.

Our findings corroborate the idea that not every test smell represents a problem to the developers, and their corresponding refactoring strategies may not be preferred in most cases, which encourages further study on whether specific test smells are meaningful to developers.

As future work, we intend to (i) increase the list of developers' validated test smells and their refactoring strategies, (ii) investigate if new test framework features could be successfully used in such refactorings, and (iii) investigate if any test smell has become obsolete due to new test framework features or properties.

ACKNOWLEDGMENTS

This work was partially funded by CAPES (117875), CNPq (421306/2018-1, 309844/2018-5, 426005/2018-0 and 311442/2019-6), and FAPEAL/FAPESP (60030.0000000462/2020) grants.

REFERENCES

[1] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and Dave Binkley. 2015. Are test smells really harmful? An empirical study. *Empirical Software Engineering* 20, 4 (2015), 1052–1094.

[2] Martin Fowler. 2018. *Refactoring: improving the design of existing code.* Addison-Wesley Professional.

[3] Vahid Garousi and Barış Küçük. 2018. Smells in software test code: A survey of knowledge in industry and academia. *Journal of Systems and Software* 138 (2018), 52–81.

[4] Nildo Silva Junior, Larissa Rocha, Luana Almeida Martins, and Ivan Machado. 2020. A survey on test practitioners' awareness of test smells. arXiv:2003.05613 [cs.SE]

[5] Stefano Lambiase, Andrea Cupito, Fabiano Pecorelli, Andrea De Lucia, and Fabio Palomba. 2018. Just-In-Time Test Smell Detection and Refactoring: The DARTS Project. (2018).

[6] Gerard Meszaros. 2007. *xUnit test patterns: Refactoring test code.* Pearson Education.

[7] Emerson Murphy-Hill, Chris Parnin, and Andrew P Black. 2011. How we refactor, and how we know it. *IEEE Transactions on Software Engineering* 38, 1 (2011), 5–18.

[8] F. Palomba, G. Bavota, M. D. Penta, R. Oliveto, and A. D. Lucia. 2014. Do They Really Smell Bad? A Study on Developers' Perception of Bad Code Smells. In *2014 IEEE International Conference on Software Maintenance and Evolution.* 101–110.

[9] Fabio Palomba, Dario Di Nucci, Annibale Panichella, Rocco Oliveto, and Andrea De Lucia. 2016. On the diffusion of test smells in automatically generated test code: An empirical study. In *2016 IEEE/ACM 9th International Workshop on Search-Based Software Testing (SBST).* IEEE, 5–14.

[10] Fabio Palomba and Andy Zaidman. 2019. The smell of fear: On the relation between test smells and flaky tests. *Empirical Software Engineering* 24, 5 (2019), 2907–2946.

[11] Anthony Peruma, Khalid Almalki, Christian D Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2019. On the distribution of test smells in open source Android applications: an exploratory study. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering.* IBM Corp., 193–202.

[12] Anthony Peruma, Christian D Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2020. An Exploratory Study on the Refactoring of Unit Test Files in Android Applications. In *Conference on Software Engineering Workshops (ICSEW'20).*

[13] Anthony Shehan Ayam Peruma. 2018. *What the Smell? An Empirical Investigation on the Distribution and Severity of Test Smells in Open Source Android Applications.* Ph.D. Dissertation.

[14] Martin Schvarcbacher, Davide Spadini, Magiel Bruntink, and Ana Oprescu. 2019. Investigating developer perception on test smells using better code hub-Work in progress. In *2019 Seminar Series on Advanced Techniques and Tools for Software Evolution, SATTOSE.*

[15] Davide Spadini, Martin Schvarcbacher, Ana-Maria Oprescu, Magiel Bruntink, and Alberto Bacchelli. [n.d.]. Investigating Severity Thresholds for Test Smells. ([n. d.]).

[16] David Thomas and Andrew Hunt. 2019. *The Pragmatic Programmer: your journey to mastery.* Addison-Wesley Professional.

[17] Michele Tufano, Fabio Palomba, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, Andrea De Lucia, and Denys Poshyvanyk. 2016. An empirical investigation into the nature of test smells. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering.* 4–15.

[18] Arie Van Deursen, Leon Moonen, Alex Van Den Bergh, and Gerard Kok. 2001. Refactoring test code. In *Proceedings of the 2nd international conference on extreme programming and flexible processes in software engineering (XP).* 92–95.

[19] Bart Van Rompaey, Bart Du Bois, Serge Demeyer, and Matthias Rieger. 2007. On the detection of test smells: A metrics-based approach for general fixture and eager test. *IEEE Transactions on Software Engineering* 33, 12 (2007), 800–817.